

Database Management & Warehouse

Requirements Collection → Conceptual Design (E-R) → Logical Design → Physical Design

* ER Model: ->

It is used to represent data clearly for both tech & Non-tech people.

Entity -> A real world object with independent existence

ex: Physical -> Student, Car, laptop

Conceptual -> Company, Course, Bank Account

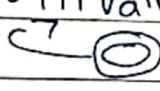
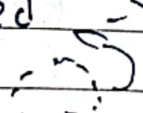
Notation: Rectangle.

Attribute -> Property that describes an entity

ex: employee -> Name, Age, Salary

Notation: Oval

Types of Attributes: -

1. Simple / Atomic -> Cannot be divided further
2. Composite -> Can be split into sub parts.
3. Multivalued -> entity that can have multiple values.

 ex: phone-numbers, degrees.
4. Derived -> Calculated from another attribute


-> Null Values

(Schema)

(State)

* Entity Type vs set

Defⁿ of entity structure Actual Data stored
ex: EMPLOYEE (Name, ID, salary) at a given time.

Keys :- (To ensure uniqueness)

'> It is distinct for each individual entity.

① Super Key :-

Attribute set that uniquely identifies an entity.

Attributes : {State, Reg. No, Serial-No, Make, Model}

Super Keys : {Serial-No} {Serial-No, Make}
{State, Reg-No} {Serial-No, Model}

② Candidate Key :-

Minimal Super Key.

ex: {Serial-No} {State, Reg-No}

③ Primary Key :- Chosen Candidate Key.

{Serial-no}

Notation : Underlined attribute.

④ Composite Primary Key :

Primary Key made of multiple attributes.

ex: Works-on (Employee-ID, Project-ID)

Together, unique

#Relationships:-

→ An association among entities.

ex: Employee works for department.

→ Relationship Instance: Association between specific entity instances.

ex: employee e1 works for department d1.

→ Relationship Type:

ex: works-for (between employee & department)

Notation: Diamond.

Degree : → Binary

→ Ternary

Recursive Relationship : - (1:N)

→ When the same entity type participates more than once in a relationship.

ex: employee supervises employee.

Cardinality Ratio:- (Maximum Constraint)

→ Defines the max^m number of Relationship Instances an entity can participate in.

1:1, 1:N, M:N

Participation Constraint:-

→ Defines whether an entity's existence depends on participating in a relationship.

Total Participation:- (Double line) (1, N)

→ every entity must participate

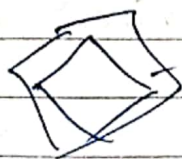
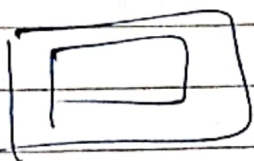
Partial Participation:- (Single line) (0, 1)

→ Some entities may not participate

Weak entity:-

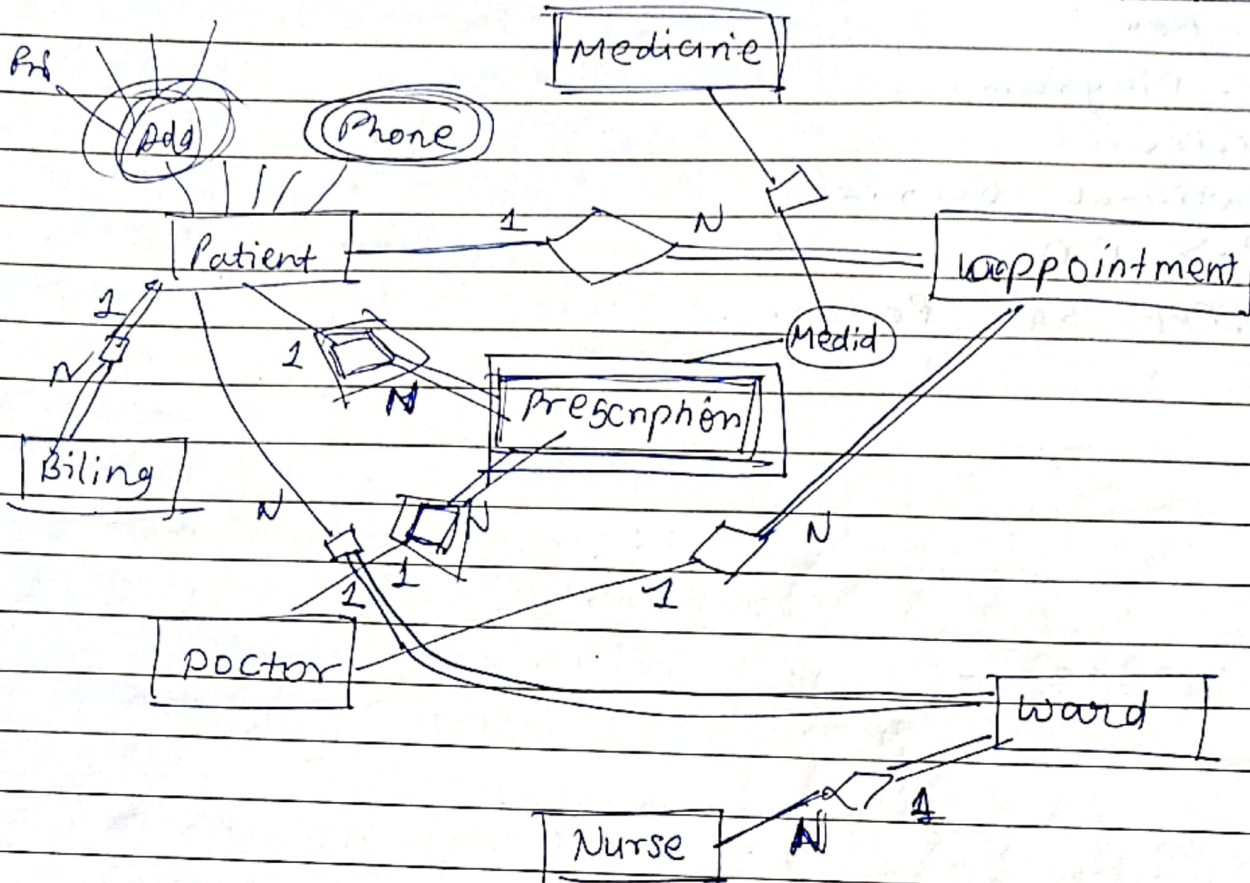
→ An entity that does not have primary key of own, depends on other entity for identification: (Strong)

→ It uses Partial Key to distinguish instances belonging to same owner.



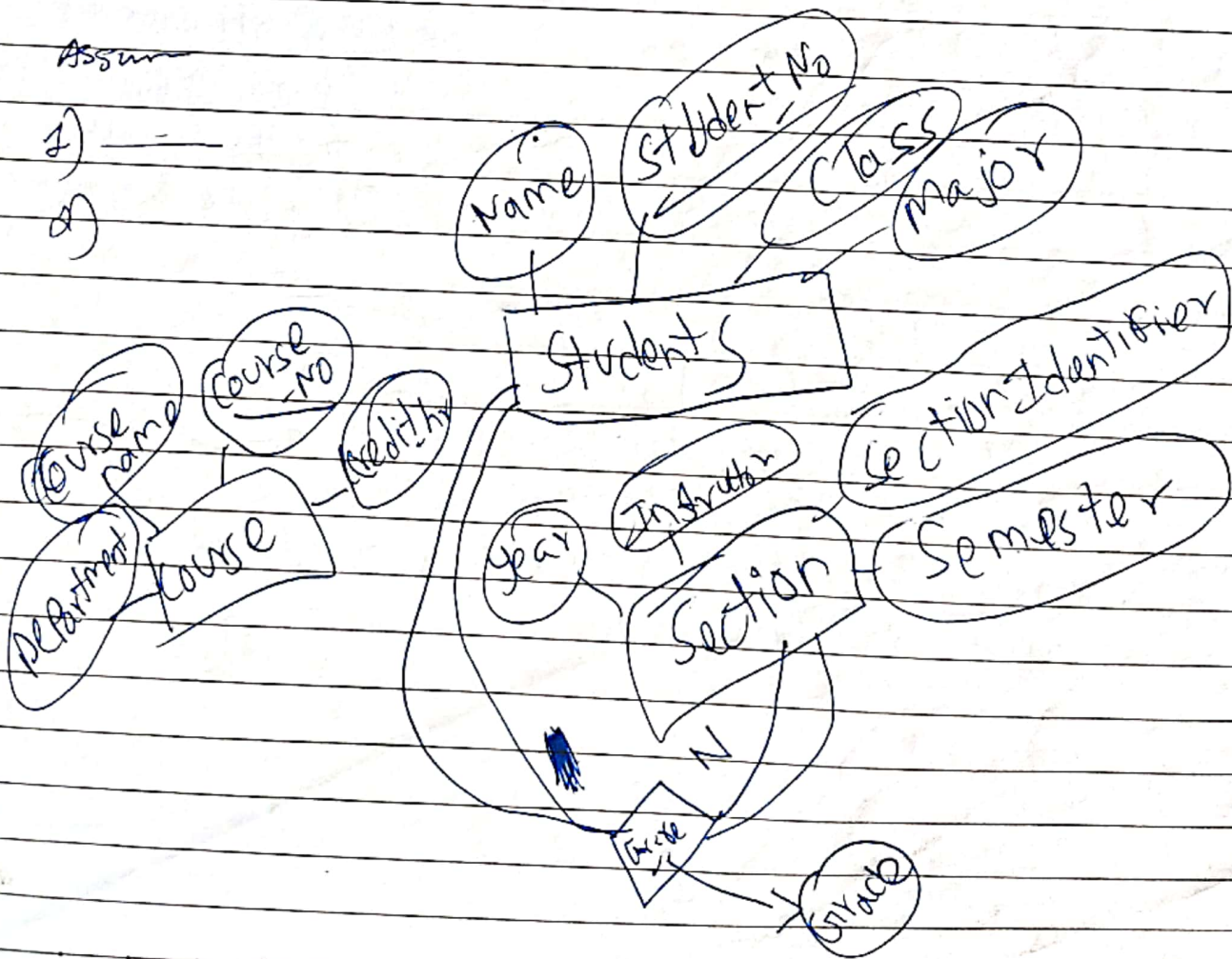
→ Identifying Relationship

E-R Diagram



Assum

- 1) —
- 2)



Relational model concepts

Relation : Table

Tuple : Row

Attribute : column

Domain : Set of valid atomic values for an attribute

Degree : no. of columns.

Cardinality : no. of rows

* Characteristics : -

→ Ordering is immaterial

→ All values are atomic, no composite / multivalued allowed in 1st NF.

→ Multivalued Attributes must be stored in separate re

→ Composite Broken into Atomic attributes.

→ No two tuples can be identical.

** Relational Integrity Constraints :-

1) Domain Constraint

2) Key constraint : $t_1 [SK] \neq t_2 [SK]$

3) Entity Integrity Constraint :-

→ Primary Key cannot be null

4) Referential Integrity Constraint.

→ If table A references table B

→ Foreign Key in A must match Primary Key in B

$$t_1 [FK] = t_2 [PK]$$

— x —

Relational Algebra:-

→ A procedural Query language

1) Select (σ) - Row filter

→ filters row Based on condition

σ condition (Relation)

Select + where (SQL)

2) Project (π) - Column filter

→ Selects specific attributes.

π Attribute 1, Attribute 2 (Relation)

Select (SQL)

3) Union → conⁿ Same number of attributes

→ Same domains

4) Intersection

5) Minus

6) Cartesian Product ($\times$)

→ Combines every row of R with every Row of S.

ex: EMPLOYEE $\times$ Department

↓

Produces all combⁿs.

7) Join operation ($\bowtie$)

→ Combines Related Tuples.

Equi-Join → Join using equality condⁿ.

ex: EMPLOYEE $\bowtie$ mgrssn = ssn Department

8) Natural Join

→ Automatically joins tables on attributes with same name.

EMPLOYEE $\bowtie$ Department.

* SQL Joins.

→ Inner Join

→ Outer Join

↳ Returns matching + non-matching Rows

Left outer join

Right outer join

→ All rows from

→ All rows from Right

Left table, if no

table, if no

match null on right

match null on left.

AV (A $\bowtie$ B)

BV (A $\bowtie$ B)

* Division Operation :-

Queries in Relational Algebra.

— X —

ER To Relational

STEP 1: Map strong entities

Attribute $\rightarrow$ Columns

Key attribute $\rightarrow$ Primary Key

STEP 2: Map weak entities

Add owner's Primary Key as Foreign Key

Primary Key = owner FK + Partial Key

STEP 3: Map 1:1 Binary Relationship.

Put Foreign Key in the relation with
Total Participation.

STEP 4: Map 1:N Binary Relationship.

Put Foreign Key in the relation
on N-side of relationship.

STEPS: Map $M:N$ Relationship.

Create New table

Add foreign Key from both entities

Primary Key = Combination of both Foreign Keys.

Add Relationship attributes if any.

STEP 6: Map Multivalued attributes.

Create separate table.

Primary Key = attribute value.

STEP 7: for composite Break into the simple attributes.

① EMPLOYEE

<u>Dnumber</u>	Fname	Minit	Lname	<u>SSN</u>	Bdate	Address	Sex	Salary
----------------	-------	-------	-------	------------	-------	---------	-----	--------

Department

<u>Dname</u>	<u>Dnumber</u>	mgr_ssn	start_date
--------------	----------------	---------	------------

Project

<u>Pname</u>	<u>Pnumber</u>	Plocation	<u>Dnumber</u>
--------------	----------------	-----------	----------------

Dependent

<u>Name</u>	Sex	Birth_date	Rel'ship	<u>ESSN</u>
-------------	-----	------------	----------	-------------

SQL: Structured Query Language.

DDL (Data Definition Language) :-

- ↳ Defines the structure
- ↳ (create, Alter, Drop, Truncate, Rename)

DML (Data Manipulation Language) :-

- ↳ Manipulates data
- ↳ (INSERT, UPDATE, DELETE, SELECT)

DCL (Data Control Language)

- ↳ Security
- ↳ (GRANT, REVOKE)

TCL (Transaction Control Language)

- ↳ Managing work
- ↳ (Commit, Roll back)

Data Types :- { INT, FLOAT, REAL, INTEGER, DECIMAL (i,j)

{ CHAR (n)
VARCHAR (n)

{ DATE (YYYY-MM-DD)
TIME (HH:MM:SS)
TIMESTAMP (DATE + Time)

Advanced SQL :->

* Triggers :->

Automatic action executed when an event occurs

Trigger Components .

1. event -> Insert / Update / Delete
2. condⁿ -> when clause
3. Action -> SQL statements.

EX: Prevent salary > Supervisor ::

```
CREATE TRIGGER Salary_violation
BEFORE INSERT OR UPDATE OF salary, supervisor_ssn
ON EMPLOYEE
FOR EACH ROW
WHEN (
```

NEW salary >

(SELECT salary FROM employee
where ssn = new.supervisor-ssn)

BEGIN

RAISE EXCEPTION 'salary exceeds';

END;

* Views :->

-> Stored Query

-> Does not Physically store Data.

(CREATE VIEW -> Then USE -> Then Prop)

Schema Modification :-

- DROP (Deletes schema object)
- CASCADE (Drop everything dependent on it.)
- Restrict (Only Drop if no dependencies exist)
- ALTER
- ADD

Normalization :-

→ Decompose table into multiple tables

1. eliminate redundancy
2. Improve data logic
3. Avoid Anomalies :-

* Anomalies :-

(i) Insertion :-

Occurs when data depends on other data in same table, so Insert data needs to keep some columns null.

ex: add new department 'Robotic' we must have student appointed to it otherwise we will put null which won't work because student_id is primary key.

(ii) Deletion :-

Deleting last employee with department Robotics, we lose info. about Robotic department.

(iii) Update anomaly :-

If department name changes then all candidates with that department has to change/update.

* $X \rightarrow Y$ if Y is a subset of X

Closure Method :-

Find all the Candidates Keys.

Ex: $R(ABCD)$

F.D. $\{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$

① $A^+ = ABCD$

$B^+ = BCD$

$C^+ = CD$

$D^+ = D$

$\therefore$ Candidate

Key = $\{A\}$

Prime Attribute = $\{A\}$

Non Prime Attribute = $\{B, C, D\}$

Ex: $R(ABCD)$

F.D. $\{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$

① $A^+ = ABCD$

$B^+ = BCDA$

$C^+ = CDAB$

$D^+ = DABC$

$\therefore C.K = \{A, B, C, D\}$

$\hookrightarrow$ Prime All

None Prime = $\{ \}$

1st missing check

Ex: $R(ABCDE)$

$\Rightarrow$ FD $\Rightarrow \{A \rightarrow B, BC \rightarrow D, E \rightarrow C, D \rightarrow A\}$

$$\Rightarrow A^+ = AB$$

$$B^+ = B$$

$$C^+ = C$$

$$D^+ = DAB$$

$$E^+ = E C$$

$$A E^+ = A E B C D$$

$$B E^+ = B E C D A$$

$$C E^+ = C E$$

$$D E^+ = D E C A B$$

$\therefore$ CK: $\{A E, B E, D E\}$

Prime: $\{A, B, D, E\}$

Non-Prime: $\{C\}$

Normal Form $\Rightarrow$

Table is in 1NF when

- 1- each column contains atomic values
- 2- column contains single type
- 3- no duplicate rows
- 4- no replicating groups.

Fix: 1. Use different table

2. Flatten the table.

X

2nd Normal Form: - Prime $\rightarrow$ Non Prime

Table is in 2NF

(i) If it is already in 1NF

(ii) Has no Partial dependency.

Partial dependency: -

1. When table has Composite Primary Key atleast 1.

2. Proper subset of composite key contains Non-Prime attribute.

If Candidate Key $\Rightarrow \{A, B\}$

Proper subset = $\{A\}, \{B\}$
prime attribute

Non Prime = $\{C\}, \{D\}$

Ex: Composite-Primary Key: $\{SSN, Pnumber\}$
Attributes: $\{Hours, Ename, Pname, Plocation\}$

(i) $\{SSN, Pnumber\} \rightarrow Hours$

No Partial Dependency

(ii) $SSN \rightarrow Ename$

Partial

(iii) $Pnumber \rightarrow \{Pname, location\}$

Partial

Include All the referring in Partial one.

Now we need to decompose the table.

T1: {SSN, Pnumber, hours} + Rest remaining columns

T2: {SSN, Ename}

T3: {Pnumber, Pname, Plocation}

Q. R (ABCDEF)

FD: {C → F, E → A, EC → D, A → B}

⇒ CK: {CE}

Prime: {E, C}

Non Prime: {A, B, D, F}

Partial: C → F, E → A

3rd Normal Form:-

→ Table must be already in 2NF

→ No Transitive dependence

$X \rightarrow Y \rightarrow \text{Prime attribute}$
 Super/Candidate Key

Any one from LHS/RHS satisfy.

ex: R (ABCD)

FD: {AB → C, C → D}

⇒ CK: {AB}

Prime: {A, B}

Non-Prime: {C, D}

No → Prime

No 3NF

ex: R (ABCD)

FD: {AB → C, D → A}

⇒ CK: {AB, BD}

Prime: {A, B, D}

Non-Prime: {C}

3NF

B-CNF :- (Boyce - Codd Normal form)

X → Y

Super Key / Candidate Key

Q. R (ABCDEF)

FD: {AB → C, C → DE, E → F, F → A}

⇒ 1NF ✓

CK: {BA, BC, BE, BF}

2NF:-

C → DE → Partial
E → F → X

P: {A, B, C, E, F}

NP: {D}

Fourth Normal Form:- (4NF)

- Deals with multivalued dependencies
- Already in BCNF
- No multivalued attribute

**Decompose into two separate tables to achieve 4NF

In all 1NF, 2NF, 3NF we split the table in smaller ones we can join them back and get exactly original table with no extra rows, no missing rows then lossless

Lossless Join Decomposition (5NF)

Steps to check:-

Given: $R(A, B, C)$ FD: $A \rightarrow B$
 $R_1(A, B)$ $R_2(A, C)$

- Check Common Attribute ex: A
- Then check $A \rightarrow B$ means $A \rightarrow (A, B)$

Condⁿ for lossless:-

$(R_1 \cap R_2) \rightarrow R_1$ or $(R_1 \cap R_2) \rightarrow R_2$
satisfies:-
 $A \rightarrow (A, B)$

Q. $R(\text{Student ID}, \text{Name}, \text{Course}, \text{Grade})$

Decomposition A
 $R_1(\text{Student ID}, \text{Name})$
 $R_2(\text{Student ID}, \text{Course}, \text{Grade})$

Decomposition B
 $R_1(\text{Student ID}, \text{Name}, \text{Course})$
 $R_2(\text{Course}, \text{Grade})$

Determine Decomposition is lossy or lossless.

⇒ for A:
 $R_1 \cap R_2 = \text{Student ID} \rightarrow$ lossless

for B:
 $R_1 \cap R_2 = \text{Course}$ but it doesn't determine any table.
 $\rightarrow$ lossy

FD from table:-
 $\text{Student ID} \rightarrow \text{Name}$
 $\text{Student ID}, \text{Course} \rightarrow \text{Grade}$

Q. $R(A, B, C) \rightarrow R_1(A, B) \ \& \ R_2(B, C)$

Ans. $R_1 \cap R_2 = (B)$

Lossy

$(1, 2) \times (2, 1) \rightarrow (1, 2, 1)$
 $(1, 2) \times (2, 2) \rightarrow (1, 2, 2)$
 $(2, 2) \times (2, 1) \rightarrow (2, 2, 1)$
 $(2, 2) \times (2, 2) \rightarrow (2, 2, 2)$
 $(3, 3) \times (3, 2) \rightarrow (3, 3, 2)$

Minimal cover:-

Step 1: $\rightarrow$ Decompose all Relation

ex: $D \rightarrow ABC$ into

$D \rightarrow A, D \rightarrow B, D \rightarrow C$

Step 2: $\rightarrow$ Take 1 relation at a time

ex: $A \rightarrow B$ assume its not there

Can $A^+ \rightarrow B$ using other relation

// if yes if no

$A \rightarrow B$ is useless remove
 $A \rightarrow B$ is useful

Step 3:

EXTRA ATTRIBUTE IN LHS.

ex: $AC \rightarrow D$

(i) Remove A once
check $C^+ \rightarrow D$ if yes then
remove A & use $C \rightarrow D$
& vice-versa $A^+ \rightarrow D$

(ii) If both no, then $AC \rightarrow D$

Q. FD: $\{A \rightarrow B, C \rightarrow B, D \rightarrow ABC, AC \rightarrow D\}$

① $A \rightarrow B, C \rightarrow B, D \rightarrow B, D \rightarrow A, D \rightarrow C$
 $AC \rightarrow D$

②

$A \rightarrow B$	A^+	Not Redundant
$C \rightarrow B$	C^+	Not Redundant
$\checkmark D \rightarrow B$	D^+	Redundant
$D \rightarrow A$	D^+	Not Redun
$D \rightarrow C$	D^+BA	Not Red
$AC \rightarrow D$	AC^+B	Not Red

③ $AC \rightarrow D$

$C^+ CB \checkmark$
 $A^+ A^+B \checkmark$

Minimal cover:

$\{A \rightarrow B, C \rightarrow B, D \rightarrow A, D \rightarrow C, AC \rightarrow D\}$

equivalence of FD: $\rightarrow$

Q. $X = \{A \rightarrow B, B \rightarrow C\}$
 $= X = \{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$

$\Rightarrow$ • IF X covers Y $Y \subseteq X$
• IF Y covers X $X \subseteq Y$

$\checkmark X$ $A^+ = ABC \checkmark$
 $B^+ = BC \checkmark$
 $C^+ = C \checkmark$

$\checkmark Y$ $A^+ = ABC \checkmark$
 $B^+ = BC \checkmark$
 $C^+ = C \checkmark$

$\therefore X = Y$

Hashing:- (Instant Access Method)

→ It is used to directly locate a record without scanning.

= Hash function: $h(K) = K \bmod M$ $O(1)$
 $K = \text{Key}$, $m = \text{no. of buckets}$ or

→ Buckets (Disk Blocks) are data stored in blocks.

Collision: $h(K_1) = h(K_2)$

Two Keys go to the same Bucket

* Internal Hashing: (Collision resolution)

→ Chaining:

When multiple Keys hash to the same bucket, Collision occurs, Chaining resolves this by creating linked list of records.

- Main Bucket: stores first record
- Contains Pointer
- Overflow Space: forms linked list
- Contains additional records

0	700	
ex: 1	50	→ 85 → 92
2		
3	73	→ 101
4		
5		
6	76	

$O(1)$ → Best case
 $O(n)$ → worst case

→ Open Addressing :-
 When collision occurs, do not use extra space.
 Find the next empty slot in the same table.

① Empty Hash table

0	50
1	70
2	76
3	
4	

② Insert first Key

ex: $50 \mod 5 = 0$

③ Insert Second Key

ex: $70 \mod 5 = 0$

Since 0 is occupied,
 move to second slot

④ ex: $76 \mod 5 = 1$

Since 1 is occupied, move to
 next free slot.

→ Multiple / Double Hashing :-
 Uses two hash functions to resolve collisions.

$$h_1(K) = K \mod 7$$

$$h_2(K) = 1 + (K \mod 5)$$

$$h_{new} = (h_1(K) + i \cdot h_2(K)) \mod 7$$

ex: 0

1	43
2	692
3	
4	
5	
6	27

① Insert 27

② Insert 43

③ Insert 692

$692 \mod 7 = 6$ → first collision

$$h_2 = 1 + 692 \mod 5 = 3$$

$$h_{new} = (6 + 1 \times 3) \mod 7 = 2$$

~~7.1~~ External Hashing: $\rightarrow$

- Data is stored in buckets which holds multiple records.
- Hash function maps Key → Bucket number not direct disk address
- Header table is used to convert Bucket number → actual disk Block address

1. Static hashing: $\Rightarrow$ private + cons:

Cons.

Main Buckets

- ① large unused space
- ② long overflow chains?

Bucke+	340	
0	460	
		Record Printer

Overflow Buckets

Diagram illustrating a bucket sort structure with two buckets and their pointers.

Bucket 1:

- Element 1: 32
- Element 2: 26
- Element 3: 9
- Record Pointer: (points to the first element of Bucket 2)

Bucket 2:

- Element 1: 22
- Element 2: 72
- Element 3: 52
- Record Pointer: (points to the first element of Bucket 1)

max records per bucket	Number of Buckets
10	10
20	5
50	2
100	1

→ Best for fixed ^{buffer} data size

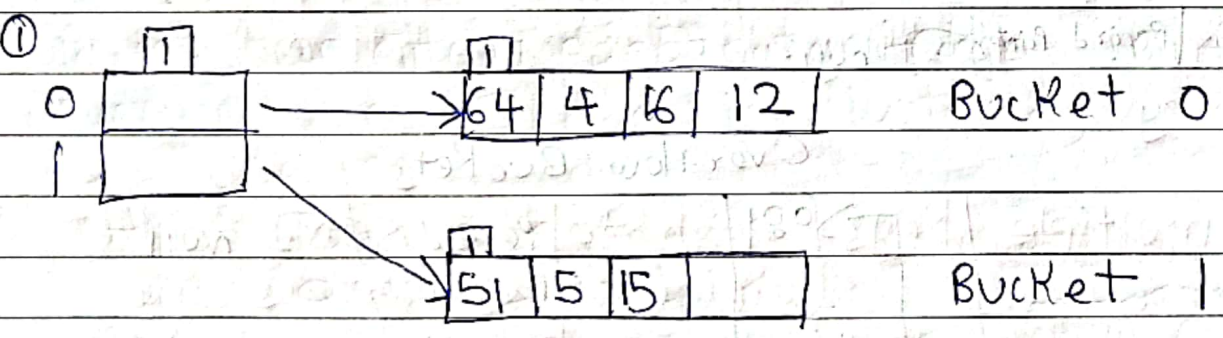
2. Extendible Hashing

- Cons: Directory too much grow if skewed data distribution
- Hash function uses LSB
 - Two depths:
 - $G.D \rightarrow$ no. of bits used by directory
 - $L.D \rightarrow$ bits used by a bucket

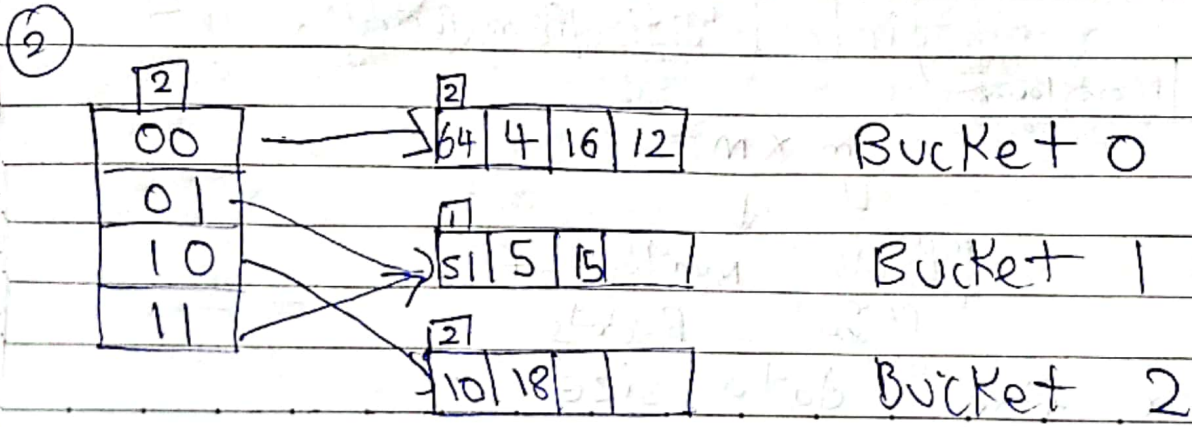
On Overflow

- Split only overflowing bucket
- Double the directory size.

Q. Insert Keys: 64, 4, 16, 51, 12, 5, 15, 10, 18, 20, 7, 21, 28 where $L.D \& G.D = 1$
 Bucket Size is 4.

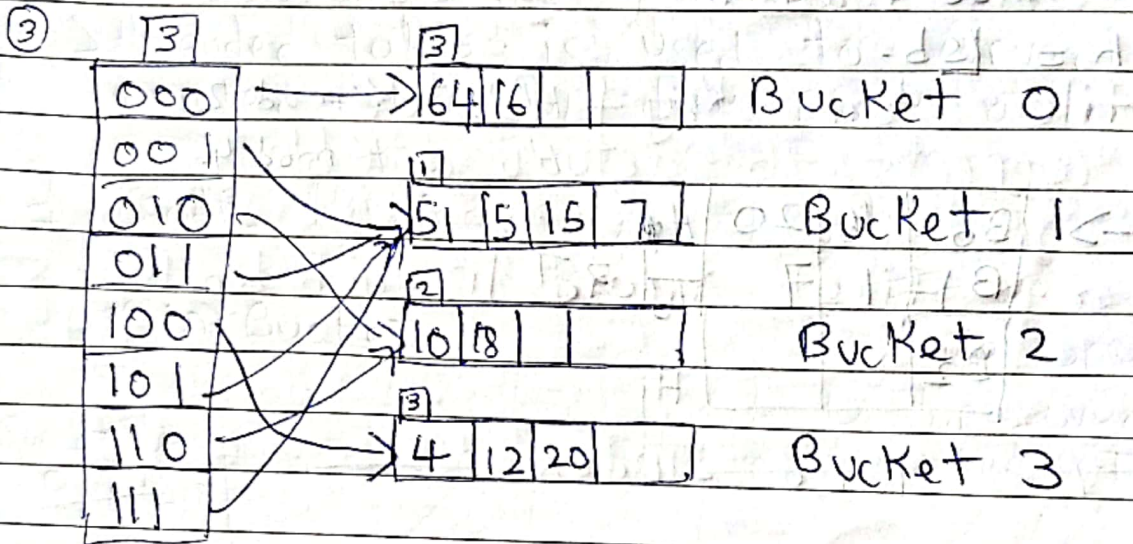


- Now when inserting 10, overflow occurs
- If $L.D = G.D$ then split directory & overflow bucket



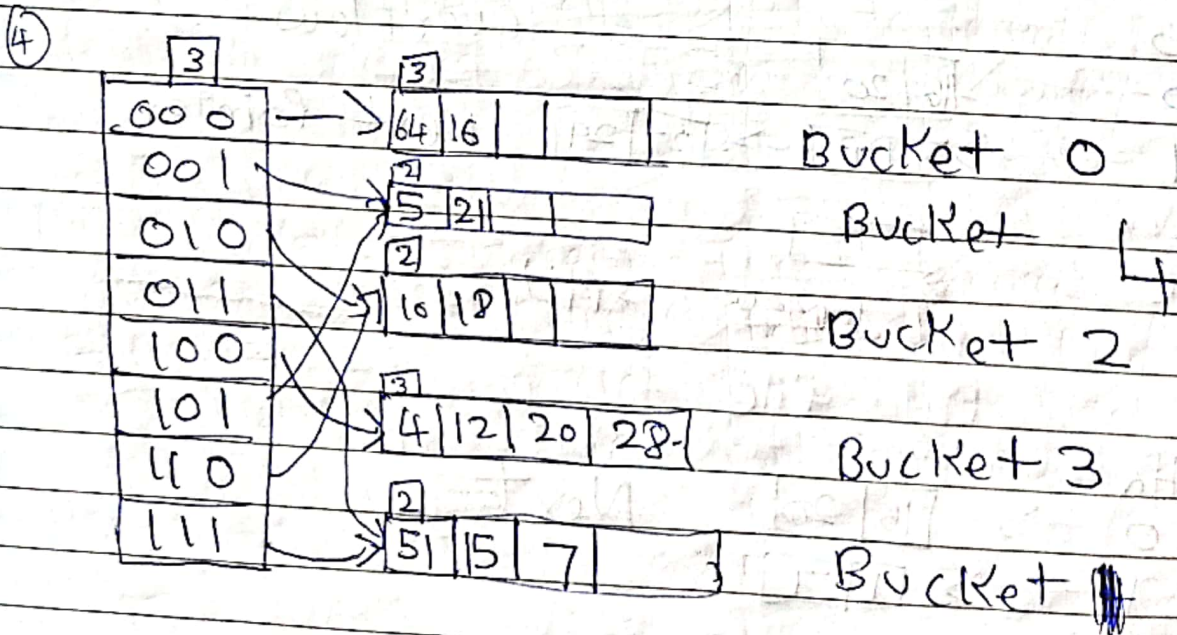
⇒ Now when Inserting 20, overflow occurs.

⇒ $LD \geq GD$ then split both directory & overflow bucket



→ Now when Inserting 21, overflow occurs in Bucket 1

→ $LD < GD$ then only split overflow bucket.



3. Linear Hashing

- Hash file expands/shrinks dynamically
- No directory used.

Q. Insert Keys: 1, 16, 20, 7, 27, 29, 18, 11, 22, 28, 9, 4
 Bucket size = 2.

Hash = $[5b]$

⇒ $H_0 = K \text{ mod } 2000$

$H_1 = K \text{ mod } 400$

Next →

B ₀	16	20
B ₁	1	7
B ₂		

H₁ 27

① H₀
 00 → [16 | 20] → next

01 → [1 | 7] → 27 overflow

② H₁ H₀ overflow

00 → [16 | 20]

01 → [1 | 7] → [27 | 29] Next Pointer

10 → []

11 → [] now split

③ H₁ H₀ Next

00 → [16 | 20]

01 → [1 | 7]

10 → []

[]

Step: 1 H_0 : initial node
 0 16 20 Next

1 11 7 → 27 overflow

Step: 2 Now split

H_1 : H_0 split
 00 0 16 20
 01 1 11 7 → 27 29 overflow Next

10 2 11 7
 11 3 11 7

Step: 3 Using H_1

H_2 : H_1 split

00 0 16 20 Next

01 1 11 29

10 2 11 7

11 3 11 7

7 27 → 11 overflow

Step: 4 Now split & use H_2

H_2 : H_1 split

000 00 16 20

001 01 11 29 Next

010 10 11 7

011 11 7 27 → 11 overflow

100 20

101 1

110 11

111 11

Database Indexing :-

- each Index entry :- $\langle K(i), P(i) \rangle$
 $K(i) \rightarrow$ Search Key value
 $P(i) \rightarrow$ Pointer (Key's disk block address)

- Index file stores Key + Pointer
- Pointer connects to actual data block.

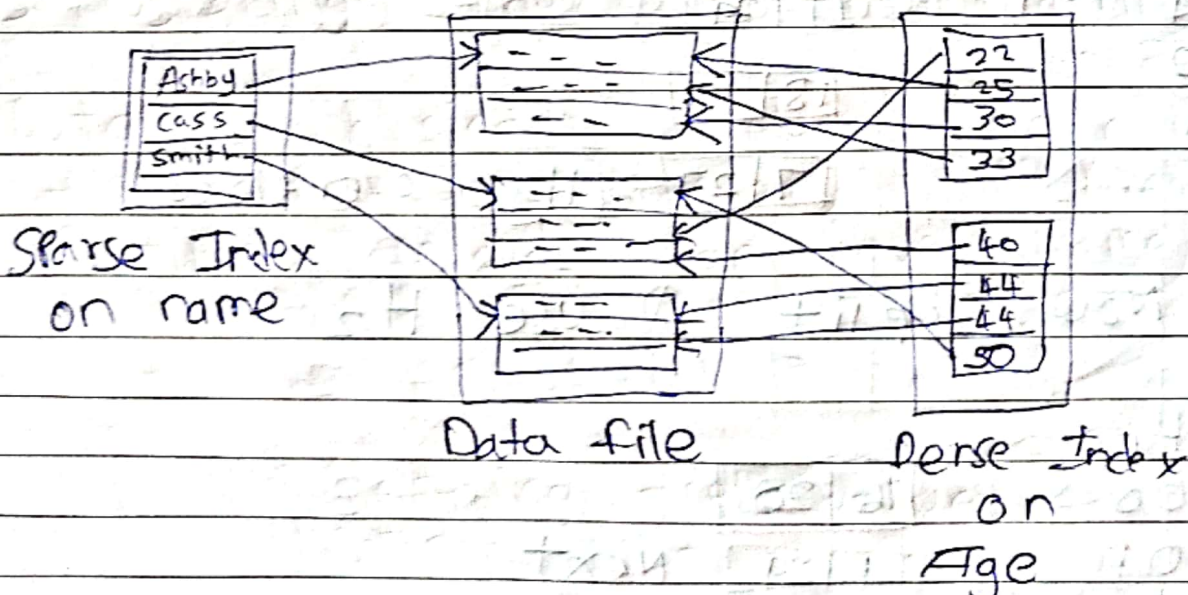
Block anchor $\rightarrow$ first record in each block.

** Dense vs Sparse Index

Dense: Atleast one data entry per Key value

Sparse: an entry per data page in file.

(every sparse Index is clustered)



* Primary Index: $\text{Index} = \text{Primary Key}$

→ Built on Primary Key

Data is ordered + unique

Density is strictly sparse

Scale: Index entries = number of disk blocks

Index file $\rightarrow$ Primary Key (Sorted)

Address of Block	Block
10	2
21	3
3F	7

1	5	6	7
10	11	14	20

Block 1

Block 2

Primary Index

Stores only
Block anchors

N Block

Q: Number of records $n = 30000$

block size $B = 1024$ bytes

Record size $R = 100$ bytes

find blocking factor, no. of blocks, block accesses using the binary search

$$\Rightarrow bfr = \left\lfloor \frac{B}{R} \right\rfloor = 10$$

$$b = \left\lceil \frac{n}{bfr} \right\rceil = 3000$$

Binary Search: $\log_2(3000) \approx 12$ block accesses

Q2. Key size $V = 9$ bytes, Pointer size $P = 6$ bytes
 = find Index entry size, Blocking factor for index

$\Rightarrow$ Index entry size $= V + P = 15$ bytes
 $bfr_i = \left\lfloor \frac{B}{15} \right\rfloor = \left\lfloor \frac{1024}{15} \right\rfloor = 68$

Q3. find no. of Index blocks,

= Total blocks accessed using index

$\Rightarrow$ Number of Index Blocks $= \left\lceil \frac{3000}{68} \right\rceil = 45$

Binary search (Index) $= \log_2 45 \approx 6$

Total access $= 6 + 1$ (Data file) $= 7$

$\rightarrow$ Clustering Index (Non-Key ordering)

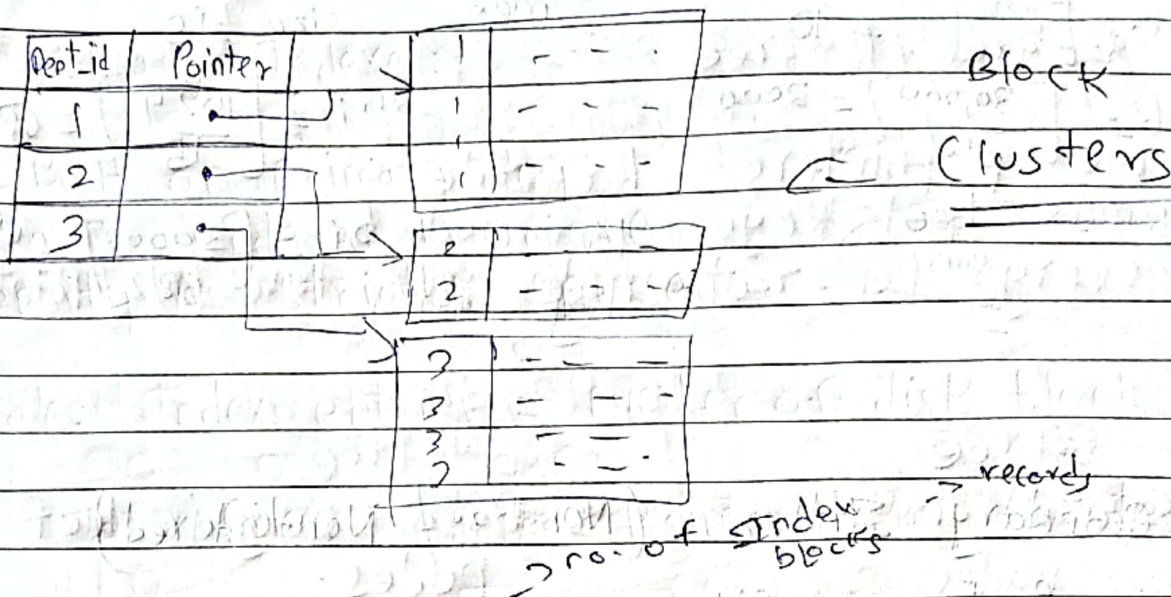
- Data is physically ordered on a non-key field (ex: Dept. number)
- Multiple records can have same value.

$\langle K(i), P(i) \rangle$

* Clustering field value

* Pointer to first block of that cluster

* Records with same value stored in the block clusters.



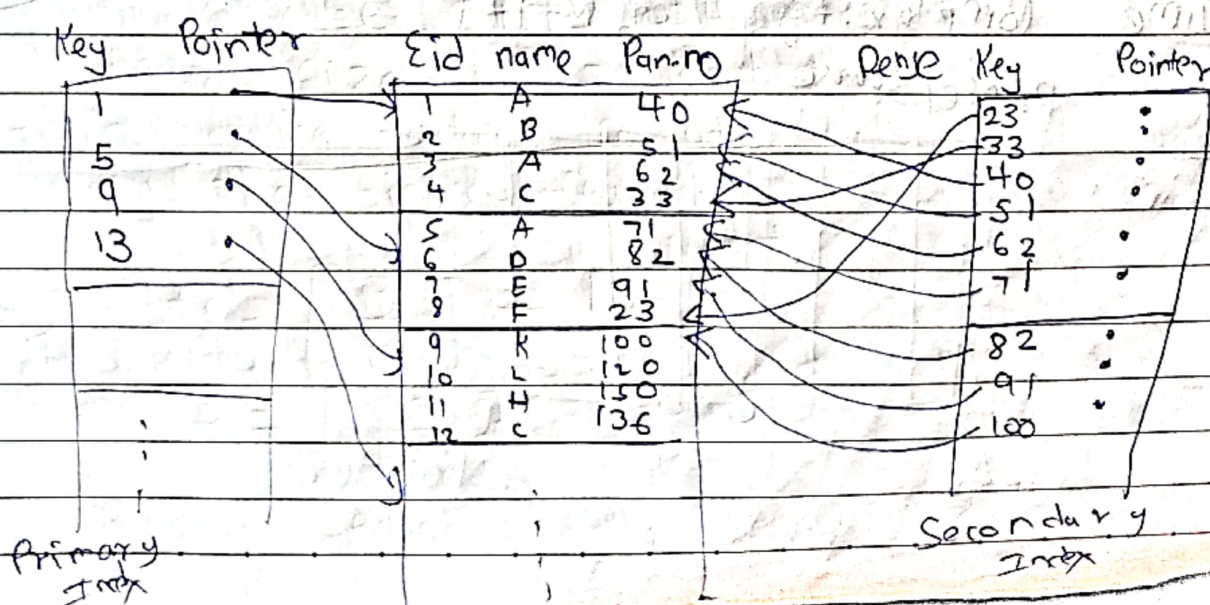
** Secondary Index (Key/Ordered field)

→ Used when data is not Physically ordered on that field.

As Data is scattered across blocks we cannot use block anchors.

∴ Index is Dense

Working: → for every record one Index entry exist
 → Index directly points to exact record location



Rough work:

$$bfr = \left\lceil \frac{100}{10} \right\rceil = 10$$

$$b = \left\lceil \frac{30,000}{10} \right\rceil = 3000$$

Index

size = 15

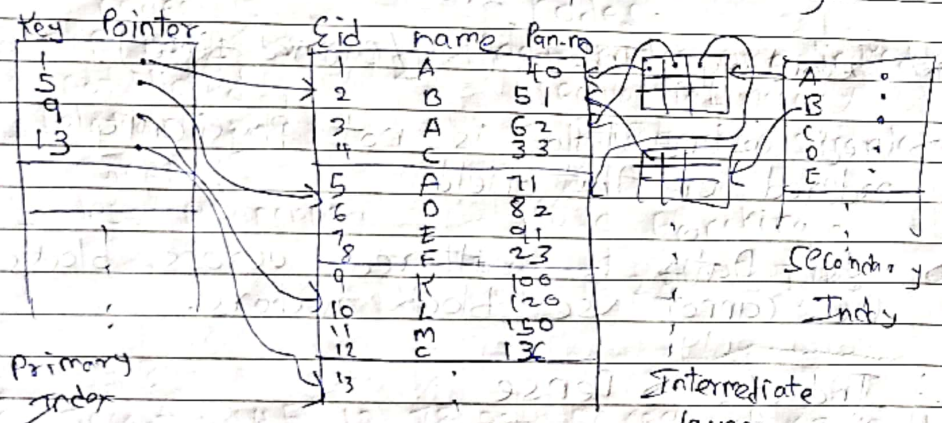
$$bfr = \left\lceil \frac{1024}{15} \right\rceil = 69$$

$$b = \frac{100}{2} = 50$$

$$\log_2(100)$$

$$bfr = \left\lceil \frac{3000}{69} \right\rceil = 44$$

Secondary Index (Non-Key / Non-ordered)



Time complexity: $\log_2 N + 1$
Pansel sparse

* Multi-level Indexing: (Indexing on Indexing)

Primary Key
fixed

A	B	C	D	E
3				
8				
12				
15				
21				
24				
29				
35				
36				
39				
41				
44				
46				
51				
52				
55				
58				
63				
66				
71				
72				

Data file

Index file: first level

Index File: Second level

A	Block Pointer
2	1
8	2
15	3
24	4
35	5
39	6
44	7
51	8
55	9
63	10
71	11
-	-

A	Block Pointer
2	1
35	2
55	3

Rough work:

STATIC
(Multilevel Index)

DYNAMIC Trees

- Structure is fixed
- Uses over flow files
- Inefficient Time Complexity

- Structure is Dynamic (grows/shrinks)
- Splits & merges the nodes.
- Maintains $O(\log n)$

B-Tree / B⁺ Tree

- max p pointers per node
- max $p-1$ keys per node
- every non-root node must have $\lceil p/2 \rceil$ pointers

BTree

- Data in all nodes
- leaf nodes are not linked
- Deletion is complex
- less Branching

B⁺ Tree

- Data only in leaf nodes.
- leaf nodes are linked (sequential access)
- Deletion is simple.
- More Branching

ETL (Extract, Transform, Load)

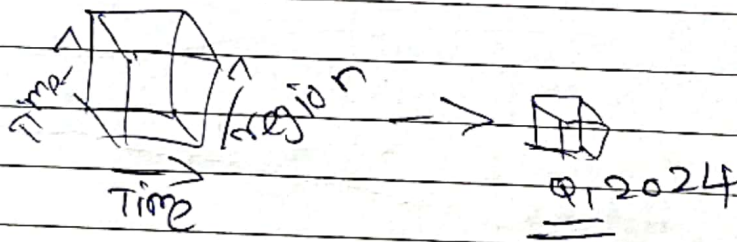
- Extract: collect data from multiple sources, can be full or incremental
- Transform: Clean, Integrate & apply rules to data (errors handle, normalization, etc..)
- Load: Processed data into warehouse tables -
↳ usually in batches.

OLAP (Online Analytical Processing)

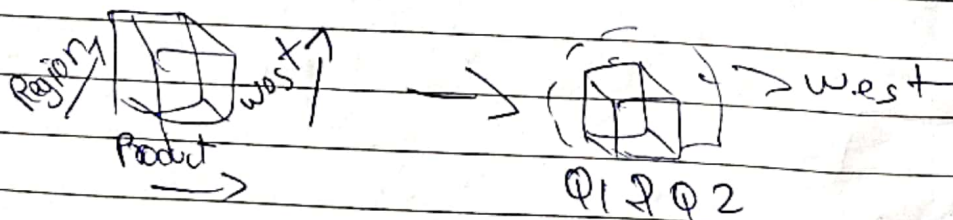
→ It refers to a set of tools used to perform fast, multi-dimensional analysis of data in warehouse.

Operations

1. Slicing → Creating a smaller cube by selecting a single dimension value

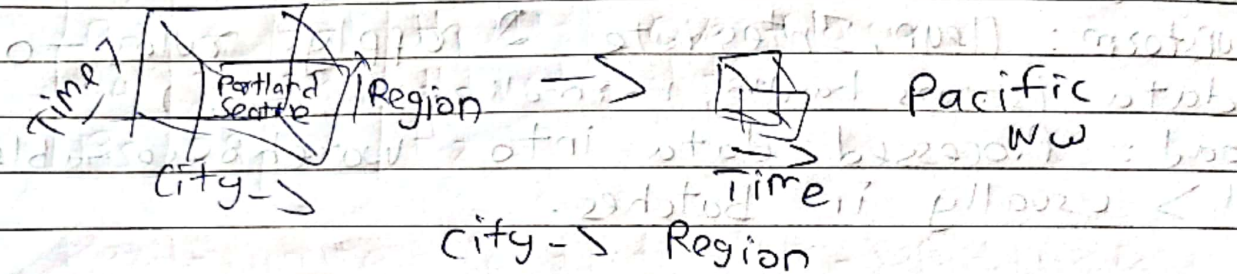


2. Dicing → Creating a sub-cube by specifying ranges for multiple dimensions



3. Roll-up (Drill-up)

→ Aggregating data by moving up a hierarchy.



4. Drill-down

→ Moving Down a Hierarchy for detailed data

ex: starting with sales by year, then drill down to see sales by Quarter, then month

5. Pivot = Reorienting the Multidimensional view of data

X

Linear Hash:

Insert 1, 16, 20, 7, 27, 29, 18, 11, 22, 28, 9, 14
Bucket size = 2

$\Rightarrow$ Initially $H_0 = \text{Key} \bmod 2$

① Next $\rightarrow$

B_0	16	20	H_0
B_1	1	7	H_0

27 overflow

Split Bucket having next pointer
 $\Rightarrow H_1 = \text{Key} \bmod 4$

②

B_0	16	20	H_1
B_1	1	7	H_0
B_2			H_1

27 29 overflow
Split

③

B_0	16	20	$H_1 \leftarrow \text{Next}$
B_1	1	29	H_1
B_2	18		H_1
B_3	7	27	H_1

11 overflow

Now again split & Increase hash

$H_2 = \text{Key} \bmod 8$

④

B_0	16		H_2
B_1	1	29	$H_1 \leftarrow \text{Next}$
B_2	18	22	H
B_3	7	27	H_1
B_4	20	28	H_2

9 overflow
(Use lowest Hash if goes in greater Hash Bucket then rehash)

X -

⑤

B ₀	16		H ₂ ← Next
B ₁	1	9	H ₂
B ₂	18	22	H ₁ ← Next - (14) overflow
B ₃	7	27	H ₁ (11) overflow
B ₄	20	28	H ₂
B ₅	29		H ₂

⑥

Next →

B ₀	16	31	H ₂
B ₁	1	9	H ₂
B ₂	18		H ₂
B ₃	7	27	H ₁ (11) overflow
B ₄	20	28	H ₂ ← Next
B ₅	29		H ₂
B ₆	22	14	H ₂

(Split again)

⑦

B ₀	16		← Next H ₂
B ₁	1	9	H ₂
B ₂	18		H ₂
B ₃	27	11	H ₂
B ₄	20	28	H ₂
B ₅	29		H ₂
B ₆	22	14	H ₂
B ₇	7		H ₂

Transaction:-

→ Single unit of work

ex: Bank Transfer → Debit + Credit both succeed.

↳ ACID properties:-

→ Atomicity: All or nothing

→ Consistency: No Rules is Violated, DB is consistent

→ Isolation: Independent executions of Transactions.

→ Durability: Data stays after commit.

Active state → Partially committed

committed state

failed

Abort

State

↳ Concurrency: Run Multiple transaction at same time, The Goal is to reduce response time & Increase Performance

Problems:-

1. Write-Write conflict.

→ Two T_s Update same data, one overwrite

Ex:

Initial balance = 1000

T_1 withdraws 200

T_2 Deposits 300

final must be 1100

but is 1300 due to

Overwrites.

2. Dirty-Read (Write-Read) Conflict.

→ A transaction reads uncommitted data from another.

→ If roll back happens, wrong data used.

Initial Balance = 1000 = I

Ex: $T_1 = I - 200 = 800$

$T_1(W)$

$T_2(R)$

T_1 Rolls back to 1000

$\therefore T_2$ uses dirty data

3. Read-write Conflict / Non-Repeatable Read

Same data read twice $\rightarrow$ Diff. values.

$\rightarrow$ Another transaction happens in between

Ex: $T_1 \rightarrow$ Query no. of employees $\rightarrow 5$

$T_2 \rightarrow$ Insert 1 employee $\rightarrow$ new employee added

$T_2 \rightarrow$ Commit

$T_1 \rightarrow$ Query no. of employees $\rightarrow 6$

4. Phantom Read

$\rightarrow$ same Query returns Diff. no. of row.

$\rightarrow$ Happens due to I/O by another T.

$I = 1000$

Ex: $T_1(R) \rightarrow I$

$T_2(W) \Rightarrow I_2 = I + 200$

T_2 commit

$T_1(R) \rightarrow I_2$

Inconsistent

Serial Schedule: One T after another

Serializability: A schedule is correct if it gives same result as serial execution

$\rightarrow$ Ensures consistency with concurrency.

R-w
w-w
w-R



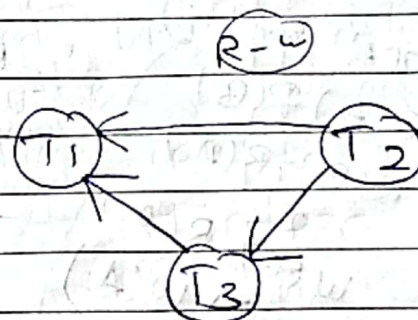
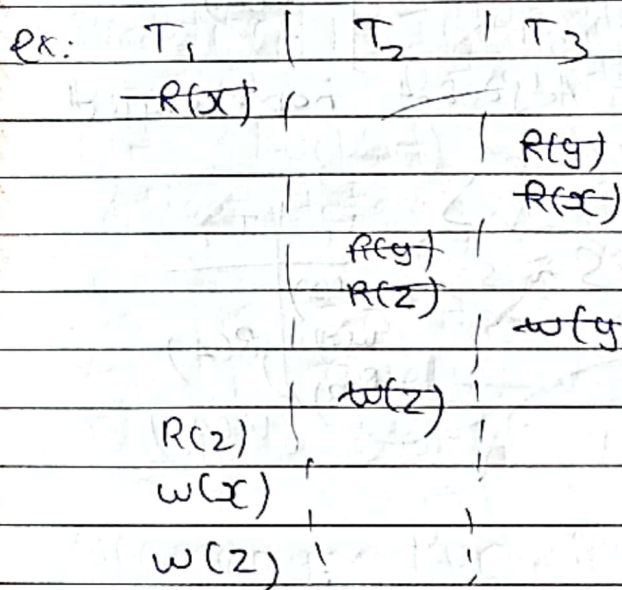
$T_1 \rightarrow T_2 \rightarrow T_3$

→ Conflict serializability.

① Check Conflict Pairs in other transactions & draw edge

R-w
w-R
w-w

Precedence graph



② Loop / cycle

In above, no cycle

∴ The Conflict is serializable schedule.

Possible cases:

③ Check vertex which Indegree = 0

$T_1 \rightarrow T_2 \rightarrow T_3$

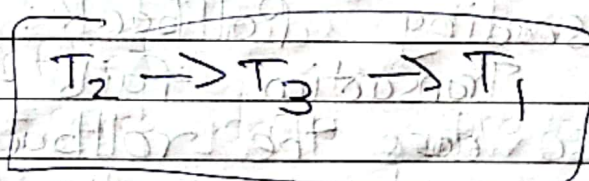
$T_1 \rightarrow T_3 \rightarrow T_2$

$T_2 \rightarrow T_1 \rightarrow T_3$

$T_2 \rightarrow T_3 \rightarrow T_1$

$T_3 \rightarrow T_2 \rightarrow T_1$

$T_3 \rightarrow T_1 \rightarrow T_2$



• If cyclic graph then non serializable

Conflict Equivalence → Data Integrity & serializability

→ Two Schedules are Conflict equivalent if they have same operations and can be converted by swapping non-conflict operation (R-R, R-w) on diff data.

→ Must Produce Same final result

R-R
 R-W → diff!

ex:

	T ₁	T ₂	T ₁	T ₂
non-conflict pairs	R(A) W(A)	R(A) W(A)	R(A) W(A) R(B)	R(A) W(A)
R(A) R(A)				
R(A) R(B)				
R(B) R(A)				
W(B) R(A)				
R(B) W(A)				

Adjacent non-conflict

T ₁	T ₂
R(A) W(A) R(B)	R(A) W(A)

T ₁	T ₂
R(A) W(A) R(B)	R(A) W(A)

Cascading Rollback:
 → One Transaction fails → Dependent transactions
 Also does the rollback.
 → Happens due to reading uncommitted data.

ex: T₁(W) X=100
 T₂(R)
 T₁: abort
 → T₂ must Rollback.

Non-Cascading Schedule:

- Transactions Read only committed data.
- No Dependency.

Ex: T1: write X = 100

T1: commit

T2: Read X

T1: Abort

T2 is Safe bcoz it read committed value

Concurrency Controls:-

A Transaction must acquire a lock on data before reading or writing.

(i) shared lock :- → only reading allowed

→ multiple transactions Allowed

(ii) exclusive lock :- → Allows both read & write
→ only one Transaction allowed.

Lock-Based Control (2PL)

Phase 1
Growing

↓
locks are acquired, no locks are released

→ locked Point

→ Phase 2
Shrinking

↓
locks are released, no locks are acquired

$T_1 \rightarrow T_2$
 conflict Serializable -

	Operation	lock acquire	lock release
Ex: T_1	Lock(X)	✓	X
T_1	Read(X)	✓	X
T_1	Lock(Y)	✓	X
T_1	Read(Y)	✓	X
T_2	Write(Y)	X	X (T_1)
T_1	Unlock(X)	✓	X
T_1	Unlock(Y)	X	✓ (shrinking)
T_1	lock(Y)	X	✓
T_2	Read(Y)	✓ (growing)	X
T_2	Write(Y)	✓	X
T_2	Unlock(Y)	X	✓

n Types of 2PL

1. Basic 2PL: follows growing + shrinking phases

→ Guarantees Conflict Serializability.

2. Conservative 2PL / Static 2PL: →

→ locks all the read items before execution

→ No deadlock, less Concurrency.

Ex: T_1 : Lock(X, Y) → R/W → Unlock(X, Y)

3. Strict 2PL: It should satisfy the basic 2PL and all exclusive locks should hold until commit/abort.
 (cascades)

ex: $T_1: lock(X) \rightarrow write(X) \rightarrow commit \rightarrow unlock(X)$
 T_2 waits until T_1 commits

* **Deadlock** Situation where two or more transactions are waiting for a lock held by the other

ex: $T_1: lock(X)$, waiting for Y
 $T_2: lock(Y)$, waiting for $X \rightarrow$ Deadlock

$\rightarrow$ Detect using wait-for graph or cycle $\Rightarrow$ Deadlock.

Prevent: older transaction gets priority.

Resolve: $\rightarrow$ Abort transaction older/new (victim)
 $\rightarrow$ Release locks $\rightarrow$ others continue.

Types of Failures

1. Transaction failure:

- logical error $\rightarrow$ overflow, divide by zero
- System error $\rightarrow$ ex: Deadlock.

2. System Crash $\rightarrow$ memory lost, disk safe

3. Disk Failure $\rightarrow$ Data lost Permanently.

Key recovery components

• System Log

→ stores all operations for recovery

log example:

[start, T₁]

[write, T₁, X, 50, 100]

[commit, T₁]

• Checkpoint

→ saves consistent state (reduces recovery time)

• Write Ahead logging (WAL)

→ log must be written before data

ex: Write log: [X: 50 → 100]

Then update disk

* UNDO → Revert to old value for aborted/incomplete

ex: T₁: X = 50 → 100 (not committed)

Crash

UNDO: X = 50

* REDO → Apply new value (for committed but not saved)

ex: T₁: X = 50 → 100 (committed)

Crash before disk write

REDO → X = 100

PRO-UNDO/REDO

→ Changes written to disk only after commit

Before commit crash → nothing to undo

After commit crash → redo needed

ex: log: T₁: [start, T₁]

[write, T₁, X, 50, 100]

[commit, T₁]

C-1: Crash before commit

T₁ → committed → Ignore → no undo

C-2: Crash after commit

redo T₁ → X = 100